```cpp
{
    return x.x < y.x;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i].x >> a[i].y;
        a[i].pos = i;
    }
    sort(a + 1, a + 1 + n, cmp);

    for(int i = 1; i <= n; i++)
    {
        int l = a[i].x, r = a[i].y;
        auto it = st.upper_bound(l);

        // 不能接在任何一个区间后面，那就新开一个区间
        if(it == st.end())
        {
            st.insert({r, ++cnt});
            a[i].num = cnt;
        }
        else // 能接在某个区间后面，那就接上去
        {
            int p = it->second;
            st.erase(it); // 把之前牛删掉
            st.insert({r, p}); // 新来的牛放进去
            a[i].num = p;
        }
    }

    cout << cnt << endl;
    for(int i = 1; i <= n; i++)
    {
        ret[a[i].pos] = a[i].num;
    }

    for(int i = 1; i <= n; i++) cout << ret[i] << endl;

    return 0;
}
```

9. 倍增思想

9.1 快速幂

> **题目来源：** 洛谷
>
> **题目链接：** P1226 【模板】快速幂
>
> **难度系数：** ★

【题目描述】

> 给你三个整数 a, b, p ，求 $a^b \bmod p$

【输入描述】

> 三个用空格隔开的整数 a, b 和 p。

【输出描述】

> 输出一行一个字符串 `a^b mod p=s` ，其中 a, b, p 分别为题目给定的值， s 为运算结果。

【示例一】

> 输入：
>
> 2 10 9
>
> 输出：
>
> 2^10 mod 9=7

【解法】

利用「二进制」以及「倍增」的思想，通过一个具体的例子说明，比如 3^{11} ：

- 幂运算的运算法则： $a^{b+c} = a^b \times a^c$ ；
- 通过一个数的二进制表示，可以写成若干数相加：
  $11 = (1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
- 两者结合： $3^{11} = 3^{(1011)_2} = 3^{1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0} = 3^8 \times 3^0 \times 3^2 \times 3^1$

因此，当我们知道 $3^1, 3^2, 3^4, 3^8, ..., 3^{log_2{}^n}$ 之后，只用做几次乘法就能计算出 3^{11} 。

如何快速算出 $3^1, 3^2, 3^3, ..., 3^{log_2{}^n}$ 。其实很简单，从前往后看，后一个数是前一个数的平方：

$$
\begin{aligned}
3^1 &= 3 \\
3^2 &= 3^1 \times 3^1 = 9 \\
3^4 &= 3^2 \times 3^2 = 81 \\
3^8 &= 3^4 \times 3^4 = 6561
\end{aligned}
$$

因此，计算 3^{11} ，我们只需将 11 的二进制表示中 1 所对应的幂乘起来即可。

如何实现这个算法呢，以 a^b 为例：

- 一边提取 b 的二进制表示中的每一位；
- 一边让 $a = a * a$，不断变成之前的平方（倍增的思想）；
- 在提取 b 的二进制表示时，如果这一位是 1，就乘上对应位置的权值。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

// a^b % p 的值
LL quickpow(LL a, LL b, LL p)
{
    LL ret = 1;
    while(b)
    {
        if(b & 1) ret = ret * a % p;
        a = a * a % p;
        b >>= 1; // 提取 b 的二进制位
    }
    return ret;
}

int main()
{
    LL a, b, p;
    scanf("%lld%lld%lld", &a, &b, &p);
    printf("%lld^%lld mod %lld=%lld\n", a, b, p, quickpow(a, b, p));

    return 0;
}
```

9.2 大整数乘法

> **题目来源： 洛谷**

> 题目链接： P10446 64位整数乘法
>
> 难度系数： ★

【题目描述】

> 求 a 乘 b 对 p 取模的值，其中 $0 \leq a,b,p \leq 10^{18}$

【输入描述】

> 第一行 a ，第二行 b ，第三行 p 。

【输出描述】

> 一个整数，表示 $a \times b \bmod p$ 的值。

【示例一】

> 输入：
>
> 2
>
> 3
>
> 9
>
> 输出：
>
> 6

【解法】

跟「快速幂」的思想一致，我们通过一个具体的例子模拟一下算法的流程，比如 3×11 ：

- 乘法的分配率： $a \times (b+c) = a \times b + a \times c$ ；
- 通过一个数的二进制表示，可以写成若干数相加：
  $11 = (1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
- 两者结合：
  $3 \times 11 = 3 \times (1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0) = 3 \times 8 + 3 \times 0 + 3 \times 2 + 3 \times 1$

因此，当我们知道 $3 \times 1, 3 \times 2, 3 \times 4, 3 \times 8, ..., 3 \times {log_2}^n$ 之后，只用做几次加法就能计算出 3×11 。

如何实现这个算法呢，以 $a \times b$ 为例：

- 一边提取 b 的二进制表示中的每一位；
- 一边让 $a = a + a$ ，不断变成之前的两倍（倍增的思想）；
- 在提取 b 的二进制表示时，如果这一位是 1 ，就加上对应位置的权值。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

// 加法比乘法快，而且防溢出
LL qmul(LL a, LL b, LL p)
{
    LL sum = 0;
    while(b) // 枚举 b 的二进制位
    {
        if(b & 1) sum = (sum + a) % p;
        a = (a + a) % p; // 计算下一个权值
        b >>= 1;
    }
    return sum % p;
}

int main()
{
    LL a, b, p;
    cin >> a >> b >> p;
    cout << qmul(a % p, b, p) << endl;

    return 0;
}
```

10. 离散化

【算法原理】

当题目中数据范围很大，但是数据的总量不是很大，并且我们需要用数据的值来映射数组的下标时，我们就可以用离散化的思想先预处理一下所有的数据，使的每一个数据都映射成一个范围较小的值。

比如：$[9, 99, 9999, 999999]$ 离散之后就变成 $[1, 2, 3, 4]$。

【离散化模板一】

代码块

```cpp
// 离散化方式一：排序 + 去重 + 二分查找离散化后的值
#include <iostream>
```